Exp No: 14	Packet sniffing using RAW sockets

AIM:

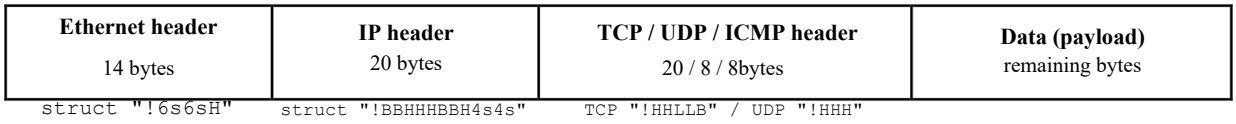
To writeaprogram using raw sockets tocapture (sniff) the packets passing through a network interface and to displaythe Ethernet, IP and TCP / UDP/ICMP header details.

ALGORITHM / PROCEDURE:

1. Create arawsocket with socket(AF_PACKET, SOCK_RAW, ntohs(3)). This socket receives every frame (ETH_P_ALL = 3) that reaches the network card, with the Ethernet header. Root permission is required.
2. Start aninfinite loop and receive a frame with recvfrom() (maximum 65535 bytes).
3. Unpackthefirst 14 bytes as the **Ethernet header**: destination MAC (6 bytes), source MAC (6 bytes) and thetypefield (2 bytes).
4. If the typeis0x0800 (IPv4), unpack the next 20 bytes as the **IP header** and get the TTL, the protocol number,thesource IP and the destination IP. The header length is found from the IHL field.
5. Using theprotocol number, unpack the transport header: 1 = ICMP (type, code), 6 = TCP (ports, sequence,acknowledgement), 17 = UDP (ports, length).
6. Displaythedetails of the packet. Repeat until the required number of packets (8) is captured.
7. Generatesome traffic in another terminal (ping, UDP echo, browser) and watch the packets being displayed.Close the socket.

Protocol number	Protocol	Header read by the program
1	ICMP	Type, Code
6	TCP	Source port, destination port, sequence no., acknowledgement no.
17	UDP	Source port, destination port, length

DIAGRAM:



Packet received by the raw socket (AF_PACKET)

Fig 1: Structure of a captured frame and the struct format used for each header



Fig 2: Flow of packet sniffing using a raw socket

PROGRAM:

sniffer.py

Name : Rohita M
Roll no : 241501168

```
# Packet sniffer using RAW sockets (Linux) - run as root
import socket, struct

def mac(b):     return ':'.join('%02x' %
x for x in b)

s = socket.socket(socket.AF_PACKET, socket.SOCK_RAW,
socket.ntohs(3)) print("Sniffing started... (capturing 8
packets)\n") count = 0 while count < 8:
    raw, info = s.recvfrom(65535)      if info[2] == socket.PACKET_OUTGOING:       # skip our  own
outgoing copy        continue     dst, src, proto = struct.unpack("!6s6sH",raw[:14])       ifproto
!= 0x0800:                    # only IPv4         continue     count +=1      ver_ihl,   tos,tlen,
ident, frag, ttl, p, chk, sip, dip = struct.unpack("!BBHHHBBH4s4s", raw[14:3        4])      ihl =
(ver_ihl & 15) * 4     print(f"Packet {count} [{info[0]}]")      print(f"      Ethernet:{mac(src)}-
> {mac(dst)}")    print(f" IPv4     : {socket.inet_ntoa(sip)} -> {socket.inet_ntoa(dip)} TTL={ttl}
Length={tlen       }")     off = 14 + ihl      if p == 1:
        t,      c      =      struct.unpack("!BB",
raw[off:off+2])            print(f"   ICMP       :
type={t} code={c}")     elif p == 6:
        sp,    dp,    seq,    ack,    offs    =    struct.unpack("!HHLLB",
raw[off:off+13])          print(f"  TCP       : {sp} -> {dp}   seq={seq}
ack={ack}")     elif p == 17:
        sp,     dp,     ln     =     struct.unpack("!HHH",
raw[off:off+6])         print(f" UDP       : {sp} -> {dp}
length={ln}")     else:          print(f" Protocol : {p}")
print()
s.close()
```

OUTPUT:

```
userUSER @userUSER -VirtualBox: ~ (Terminal 1)
```

```
userUSER@userUSER -VirtualBox:~$ sudo python3 sniffer.py
Sniffing started... (capturing 8 packets)

Packet 1 [lo]
  Ethernet : 00:00:00:00:00:00 -> 00:00:00:00:00:00
  IPv4     : 127.0.0.1 -> 127.0.0.1 TTL=64 Length=37
  UDP      : 43740 -> 6000 length=17

Packet 2 [lo]
  Ethernet : 00:00:00:00:00:00 -> 00:00:00:00:00:00
  IPv4     : 127.0.0.1 -> 127.0.0.1 TTL=64 Length=37
  UDP      : 6000 -> 43740 length=17

Packet 3 [lo]
  Ethernet : 00:00:00:00:00:00 -> 00:00:00:00:00:00
  IPv4     : 127.0.0.1 -> 127.0.0.1 TTL=64 Length=41
  UDP      : 43740 -> 6000 length=21

Packet 4 [lo]
  Ethernet : 00:00:00:00:00:00 -> 00:00:00:00:00:00
  IPv4     : 127.0.0.1 -> 127.0.0.1 TTL=64 Length=41
  UDP      : 6000 -> 43740 length=21

Packet 5 [lo]
  Ethernet : 00:00:00:00:00:00 -> 00:00:00:00:00:00
  IPv4     : 127.0.0.1 -> 127.0.0.1 TTL=64 Length=52
  ICMP     : type=8 code=0

Packet 6 [lo]
  Ethernet : 00:00:00:00:00:00 -> 00:00:00:00:00:00
  IPv4     : 127.0.0.1 -> 127.0.0.1 TTL=64 Length=52
  ICMP     : type=0 code=0

Packet 7 [lo]
  Ethernet : 00:00:00:00:00:00 -> 00:00:00:00:00:00
  IPv4     : 127.0.0.1 -> 127.0.0.1 TTL=64 Length=52
  ICMP     : type=8 code=0

Packet 8 [lo]
  Ethernet : 00:00:00:00:00:00 -> 00:00:00:00:00:00
  IPv4     : 127.0.0.1 -> 127.0.0.1 TTL=64 Length=52
  ICMP     : type=0 code=0
```

userUSER @userUSER -VirtualBox: ~ (Terminal 2 - traffic generated)

```
userUSER   @userUSER -VirtualBox:~$ python3 echo_udp_server.py &
userUSER   @USERuser -VirtualBox:~$ python3 echo_udp_client.py
Sent     : Hello UDP
Echoed   : Hello UDP
Sent     :  Datagram  test
Echoed   : Datagram test
userUSER
        @USERuser -VirtualBox:~$ sudo python3 myping.py 127.0.0.1
PING 127.0.0.1 (127.0.0.1) 32 bytes of data.
32 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.015 ms
```

Packet	Protocol	Source	Destination	Details
1, 3	UDP	127.0.0.1 : 43740	127.0.0.1 : 6000	Datagram from the client (17 and 21 byte payload)
2, 4	UDP	127.0.0.1 : 6000	127.0.0.1 : 43740	Echo from the server
5, 7	ICMP	127.0.0.1	127.0.0.1	Type 8 - Echo Request
6, 8	ICMP	127.0.0.1	127.0.0.1	Type 0 - Echo Reply

The capture was taken on the loopback interface `lo`, so the MAC addresses are all zeros. On a real interface (enp0s3) the actual MAC addresses are shown.

RESULT:

Thus a packet sniffer was implemented using raw sockets. The Ethernet, IP and UDP / ICMP header details of the packets passing through the interface were captured and displayed successfully.